

实验4实验报告

实验内容

1. 4位先行进位加法器 CLA

实验内容

几乎所有算术运算都要用到ALU或加法器，而ALU的核心还是加法器，因此要提高计算机的运算速度，关键在于提升加法器的执行速度。为了提高加法器的速度，必须尽量避免进位之间的依赖关系和传递关系，先行进位（也称超前进位）部件就能实现这一功能。

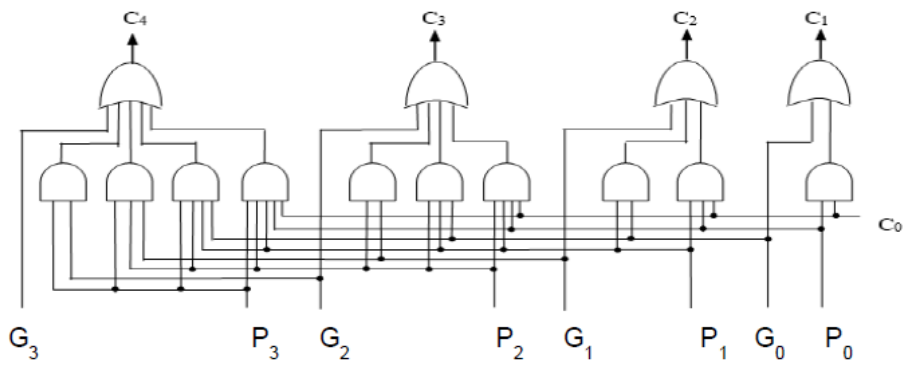
整体方案设计

输入输出引脚：

- X和y：输入的被加数和加数
- cin：输入的初始进位，在加法中默认为0
- Z：输出结果，即X+y的低四位
- Cout：加法的最高位进位
- Pg：组间进位传递函数， $P_g = P_3P_2P_1P_0$
- Gg：组间进位生成函数， $G_g = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$

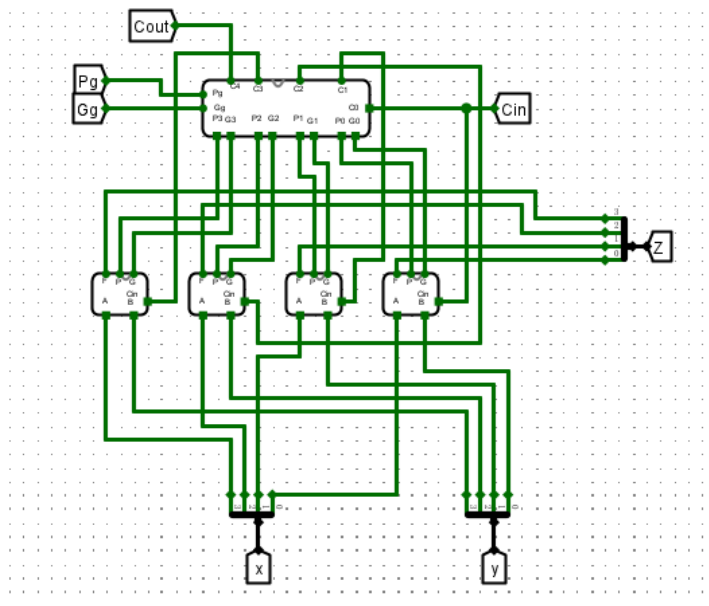
原理图和电路图

四位CLU原理图如下：

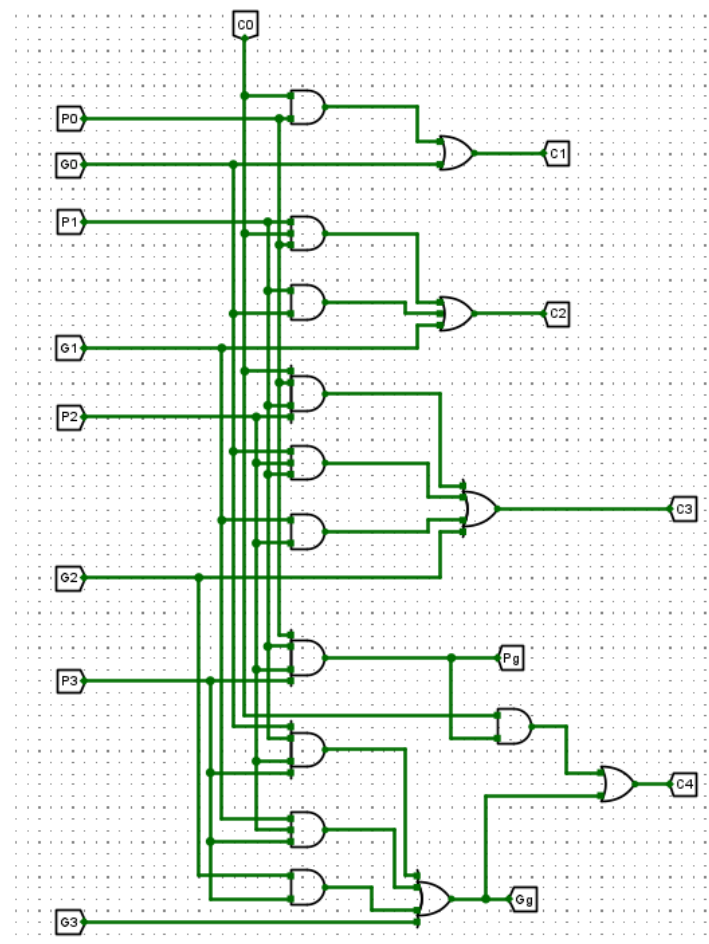


有了四位CLU后，只需把X和y每一位的FA加法结果输入到CLU中，即可得到最终结果。

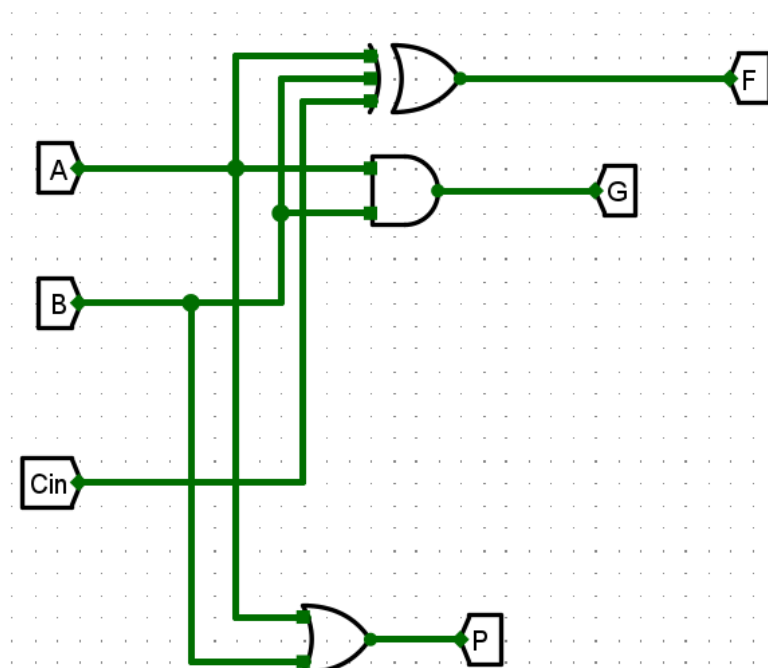
主电路图：



CLU电路图：

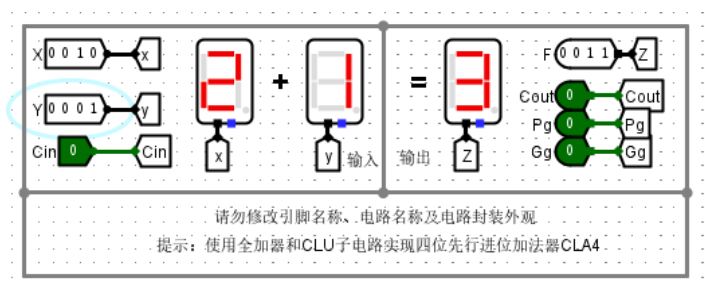


FA电路图：

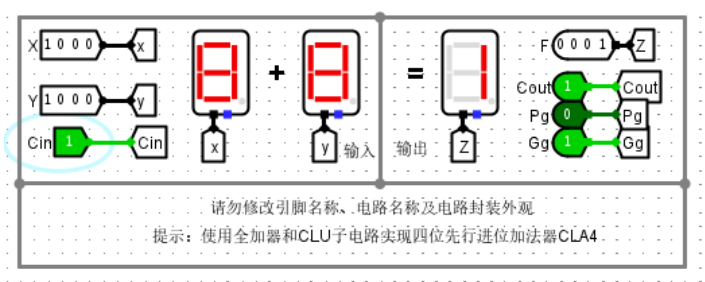


仿真测试图

X=2, Y=1, 得Z=3, 正确



X=Y=8, cin=1, 得Z=1, cout=1,Gg=1, 正确。



错误分析

在完成实验的过程中，没有遇到任何错误。

2. 16位两级先行进位加法器实验

实验内容

根据图4.8所示的原理图，在4.1的基础上，设计实现一个16位两级先行进位加法器电路，并对加法器的功能进行验证。

整体方案设计

输入输出引脚：

- X和Y：输入的16位加数和被加数
- C0：初始进位
- S：最终输出的加法结果
- C16：最高位的进位
- Pg：成组进位传递函数
- Gg：成组进位生成函数

原理图和电路图

原理图如下：

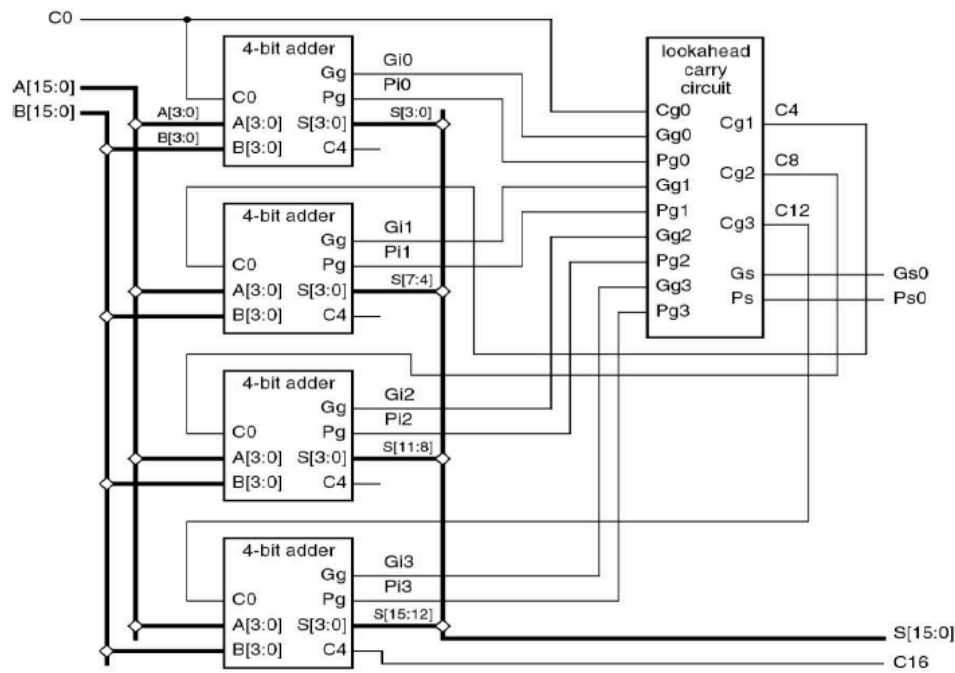
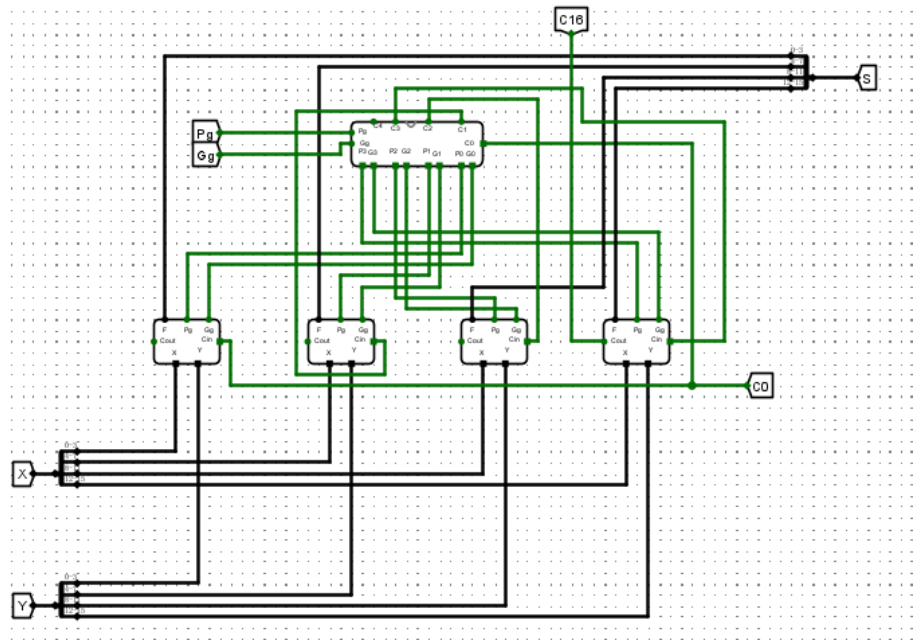


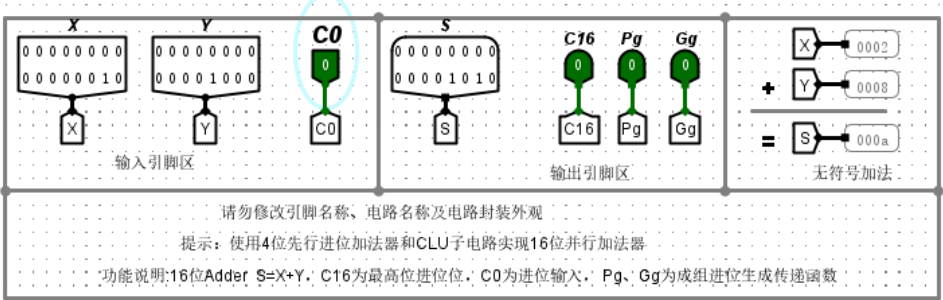
图 4.8 16 位两级先行进位加法器原理图

电路图如下：

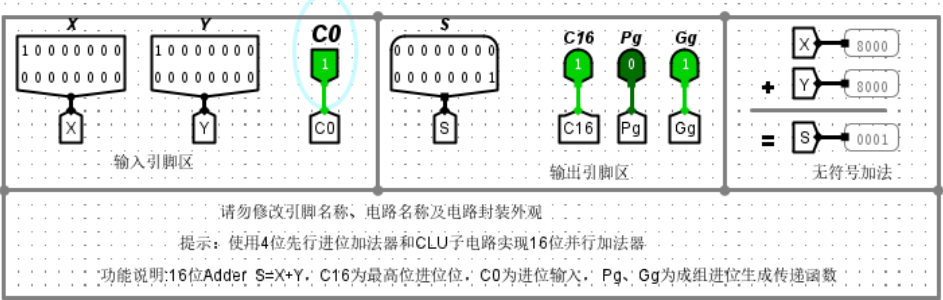


仿真测试图

X=2, Y=8, 得S=a, 正确



X=Y=8000, C0=1, 得S=0001, C16=Gg=1, 正确



错误分析

在完成实验的过程中，没有遇到任何错误，但是在最后提交的时候遇到了平台测试结果和本地不一致的现象，询问助教后发现有是电路封装不知为何出现了移动，修改电路封装和初始一致后得到解决。

3. 32位快速加法器构建实验

实验内容

通过将两个16位两级先行进位加法器串行级联构建一个 32 位加法器，并根据给出的标志位生成电路原理图，在 32位加法器中生成 CF、SF、OF、ZF标志位。

整体方案设计

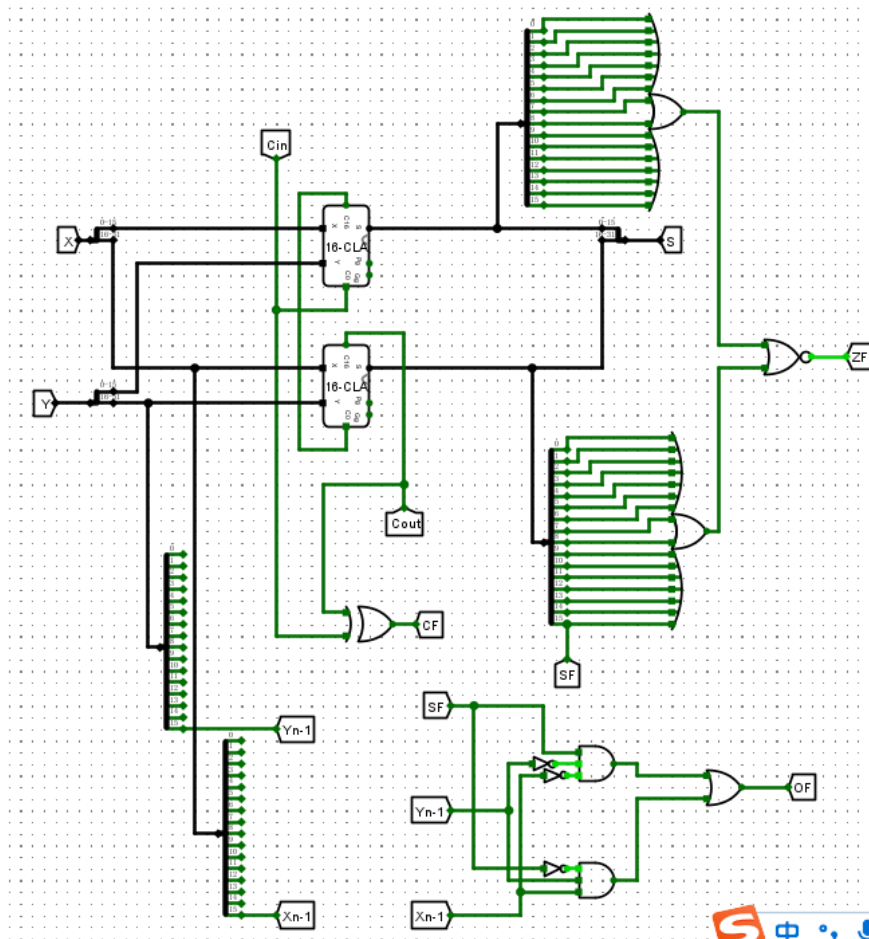
输入输出引脚：

- X和Y：输入的32位数据，代表加数和被加数
- Cin：初始进位
- S：输出的32位加法结果
- Cout：最高位进位
- CF,SF,OF,ZF：四个标志位。分别代表：进位，符号，溢出，零位

原理图和电路图

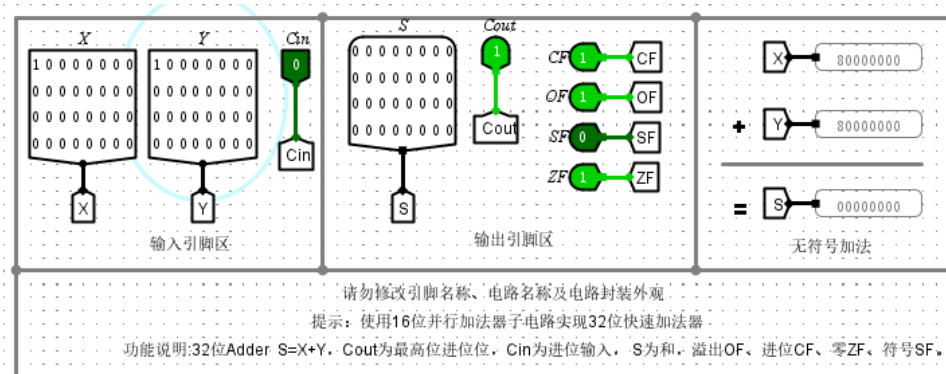
设计思路：由于4.2已经实现了16位加法器，因此32位的实现较为简便，只需级联两个16位加法器，对进位等做一些单独处理，并根据最终结果设置标志位即可。

实验电路图如下：

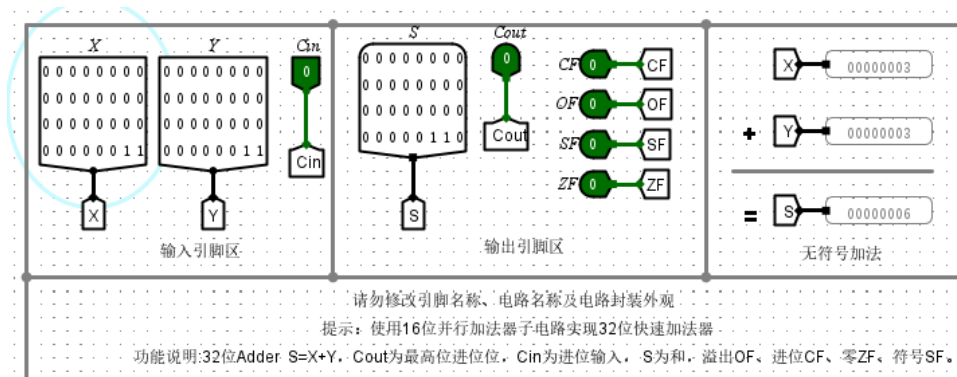


仿真测试图

$X=8000000H$, $Y=8000000H$, 则 $S=0$, $cout=CF=OF=ZF=1$, 正确。



$X=Y=3$, 则 $S=6$, 标志位均为0, 正确



错误分析

在完成实验的过程中，没有遇到任何错误。

4. 32位桶型移位器设计

实验内容

将实验2中的8位桶形移位器的电路图扩展中32位输入数据，移位位数扩展到 5位二进制数，多路选择器增加两级，分别表示移动8位和16位。

整体方案设计

输出输入引脚：

选择端L/R表示左移和右移，选择端A/L表示算术、逻辑和循环移位。（L/R,A/L）赋值为00时表示逻辑右移，赋值为01表示算术右移，赋值位10时表示逻辑左移，赋值位11时表示循环左移。

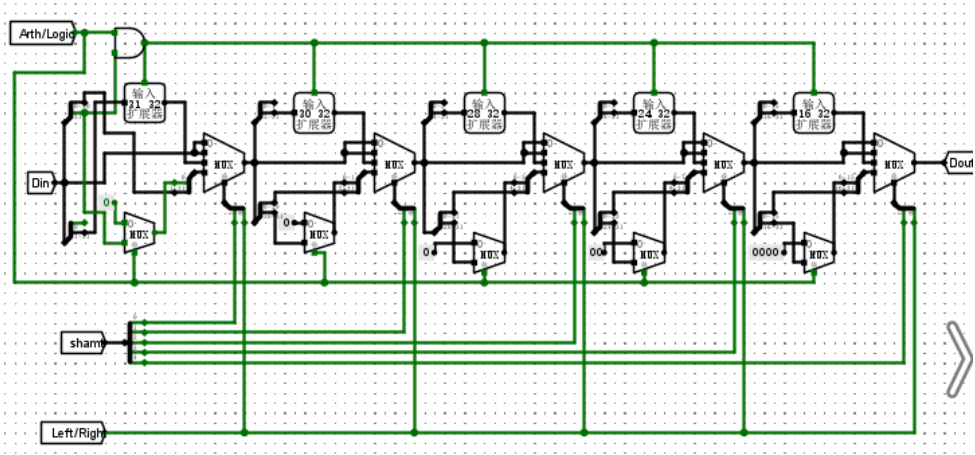
移位位数shamt为5位，代表要移位的位数

输入数据din和输出数据dout均为32位，代表移位之前和之后的数据。

原理图和电路图

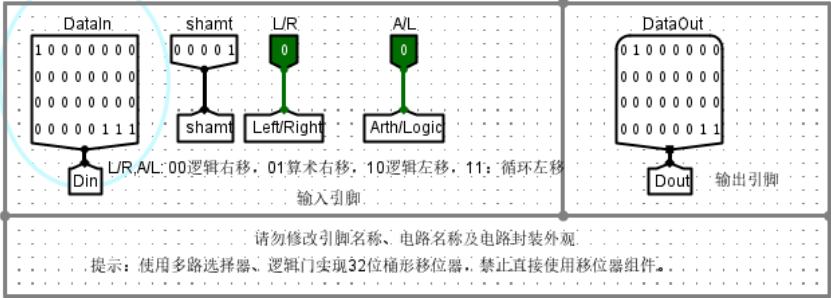
实验原理图和lab2中8位的移位器原理图类似，只是增加了两个多路选择器表示移动八位和移动16位

电路图如下：

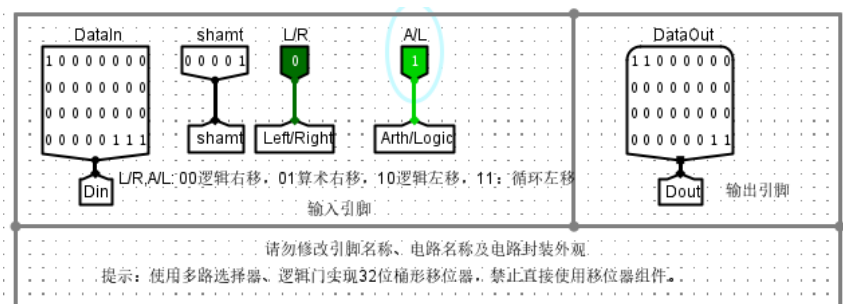


仿真测试图

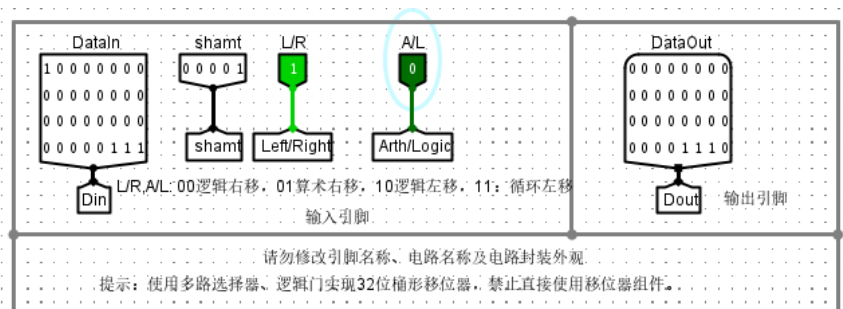
逻辑右移：



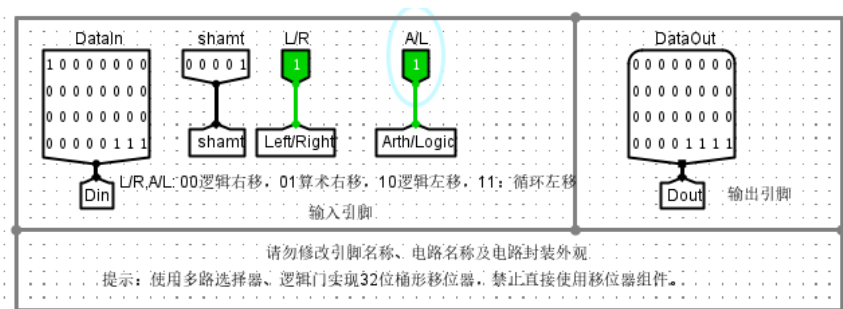
算术右移：



逻辑左移：



循环左移：



错误分析

在完成实验的过程中，没有遇到任何错误。

5. ALU设计实验

实验内容

组成 32 位 ALU 的基本部件除了基本的逻辑门、多路选择器和扩展器外，较复杂的子电路包括一个 32 位加法器、32 位移位器和一个 ALU 操作控制部件。前述实验已经设计了一个 32 位加法器和 32 位桶形移位器并封装生成了相应的子电路，因此，还需要设计一个 ALU 操作控制部件，并将其封装成子电路。最后在这些的基础上实现 ALU

整体方案设计

输入输出引脚：

- A, B: 32位输入操作数
- ALUctr: 4位控制信号
- Result: 输出32位结果
- Zero: 0标志位

原理图和电路图

原理图如下：

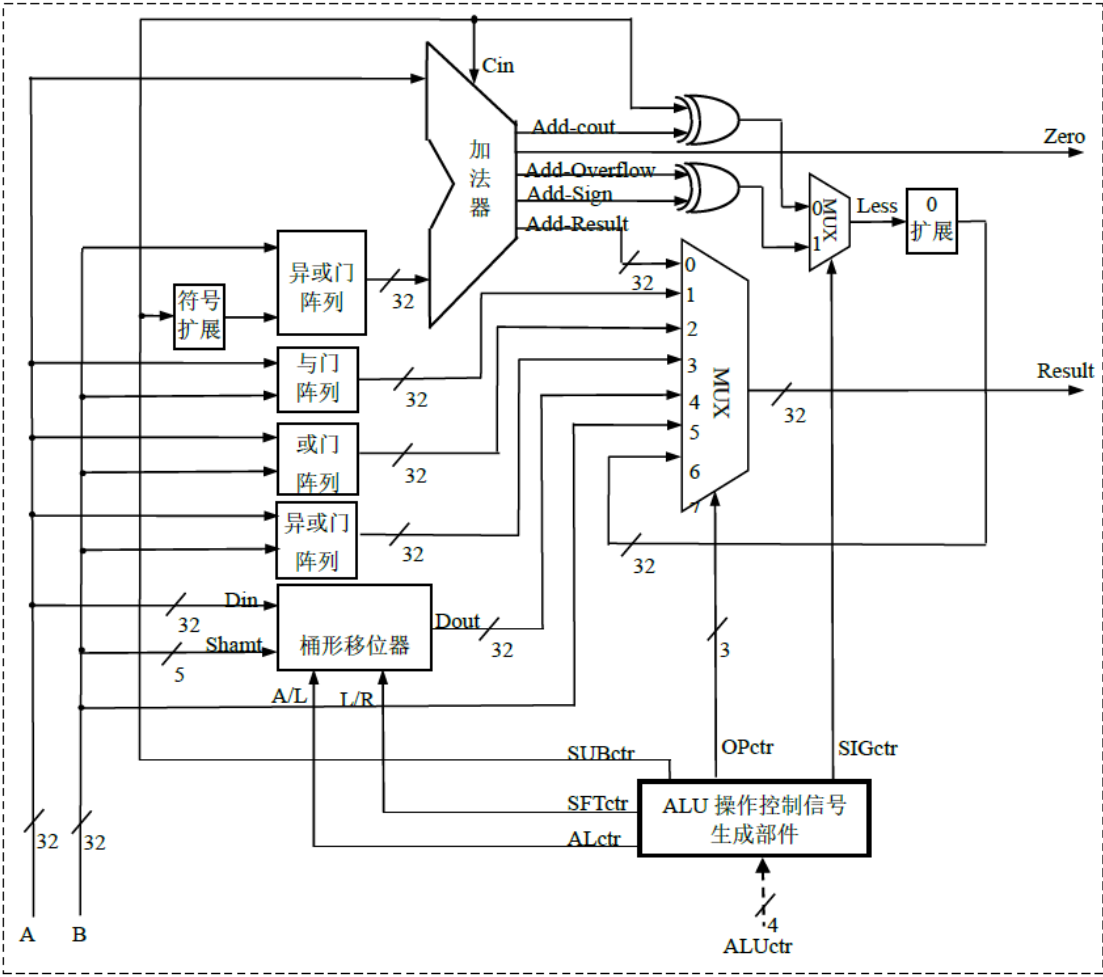
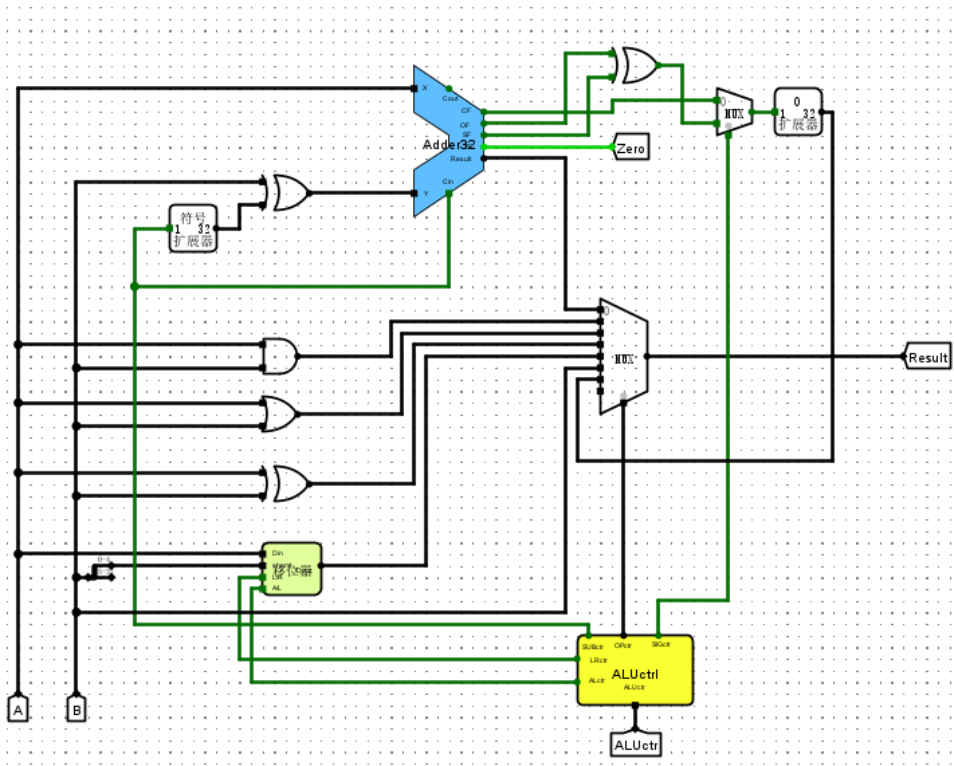


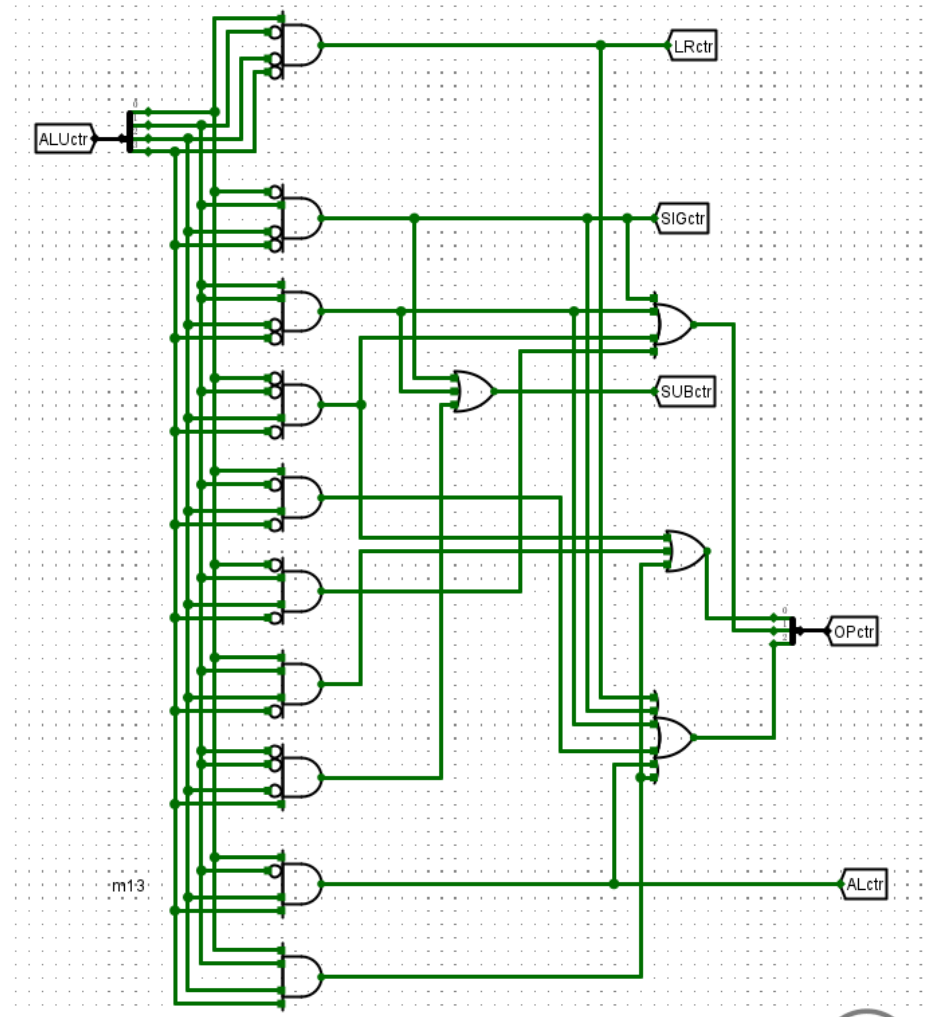
图 4.14 ALU 原理图

主电路图：



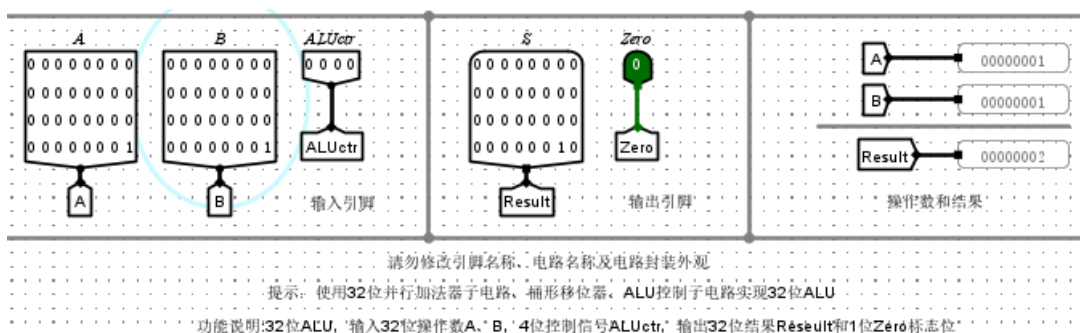
操作控制部件电路图：

ALU操作控制信号生成部件

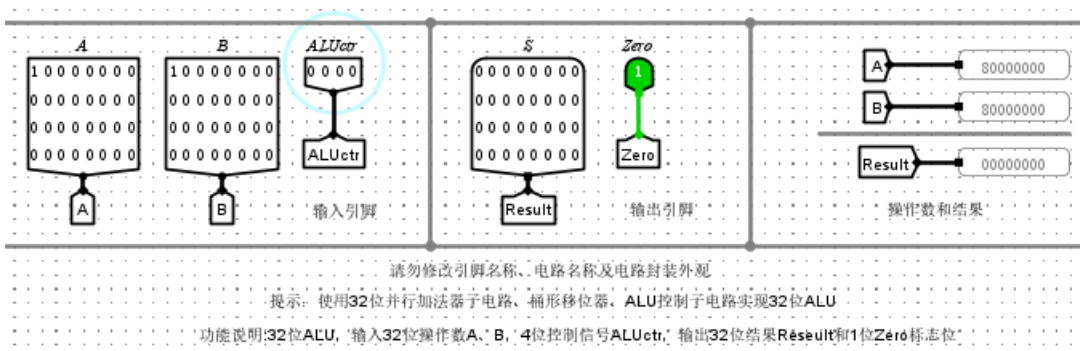


仿真测试图

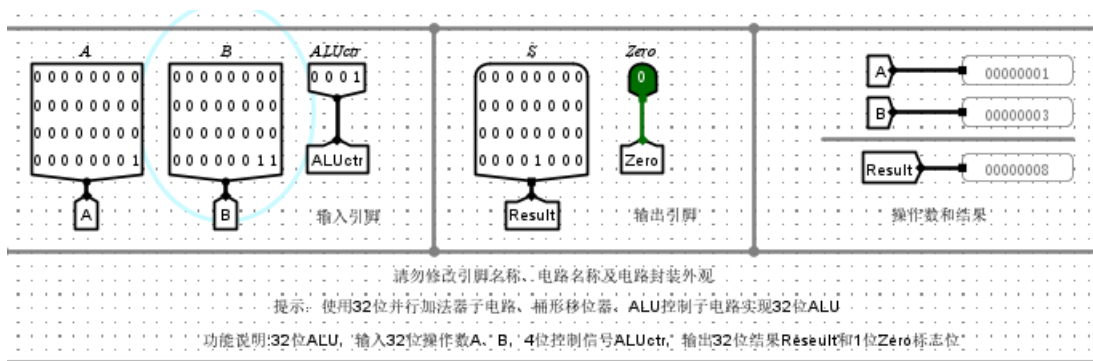
A=B=1, ALUctr=0000, 为加法运算, 输出2。



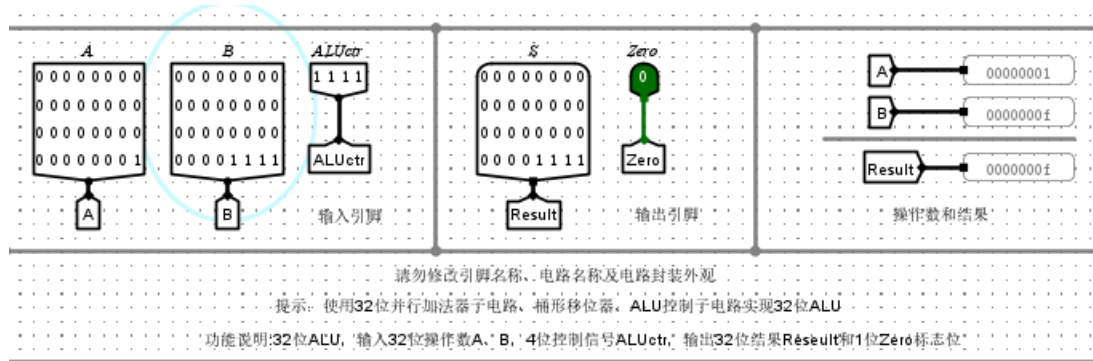
A=B=80000000H, ALUctr=0000, 为加法运算, 发生高位进位, 输出0, zero=1



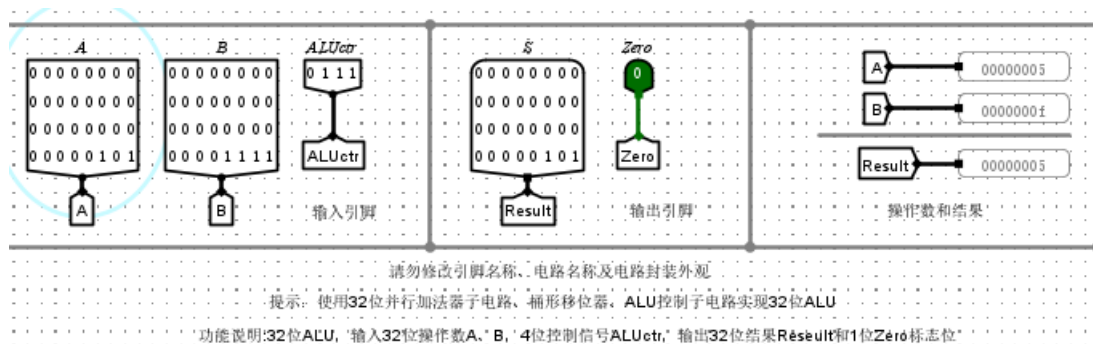
A=1, B=11b, ALUctr=0001, 为逻辑左移, 左移位数为B的低五位, 即3, 故输出result=8



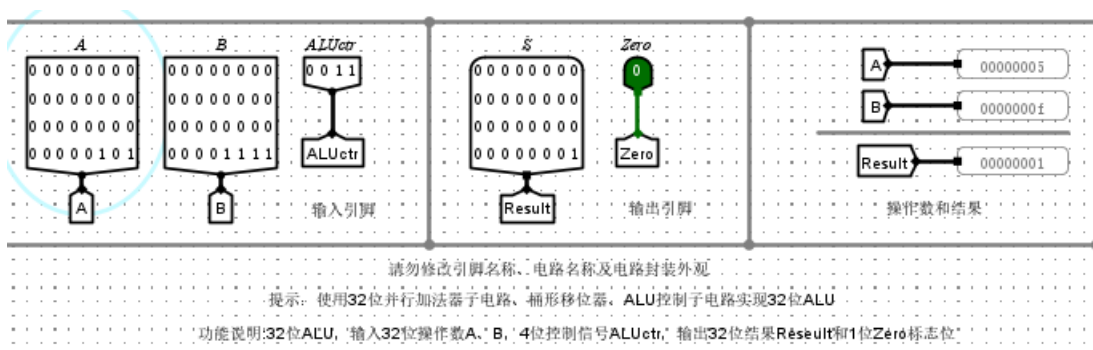
B=1111b,ALUctr=1111, 为取数操作, 输出B



ALUctr=0111, 为逻辑与操作



ALUctr=0011,为无符号数比较大小操作, 输入的A<B, 故输出1



错误分析

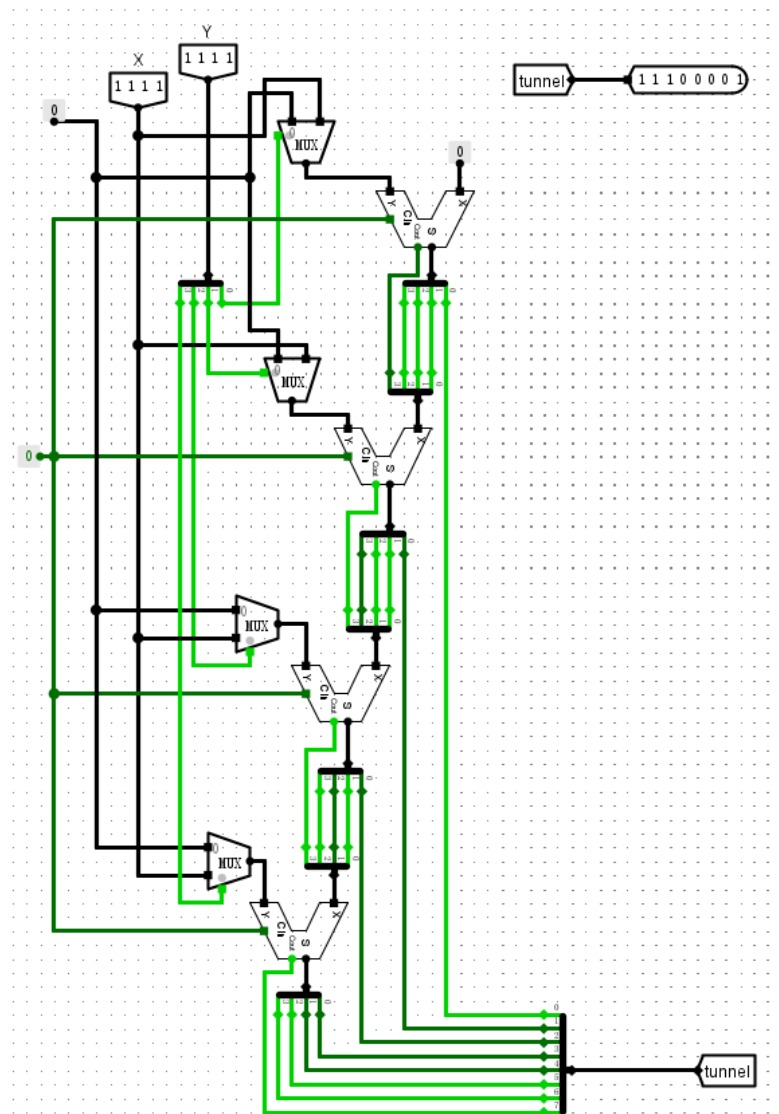
在完成实验的过程中, 没有遇到任何错误。

思考题

T1

快速乘法器基本思想：

- 为乘数的每一位提供一个n位加法器
- 每个加法器的两个输入端：本次乘数对应的位与被乘数相与的结果，和上次的一部分积
- 每个加法器的输出也分为两部分：和的最低有效位(LSB)作为本位乘积，进位和高31位的和数组成一个32位数作为本次部分积。
- 我实现了一个4位版本的，作为原理示意，保存在快速乘法器文件中，32位的实现方法是完全类似的

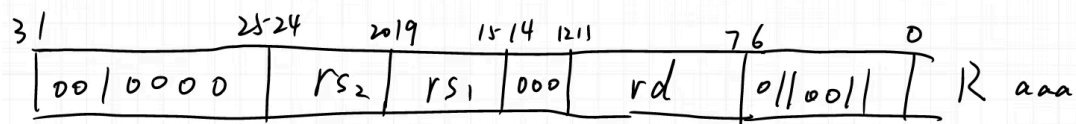


融合到ALU中：可以将乘法的结果的低32位，连接到LAB5中ALU的多路选择器的最后一个选择端，再在ALUctr中增加一个对应关系，如ALUctr=1100时，OPctr为111，选择乘法的结果输出。

T2

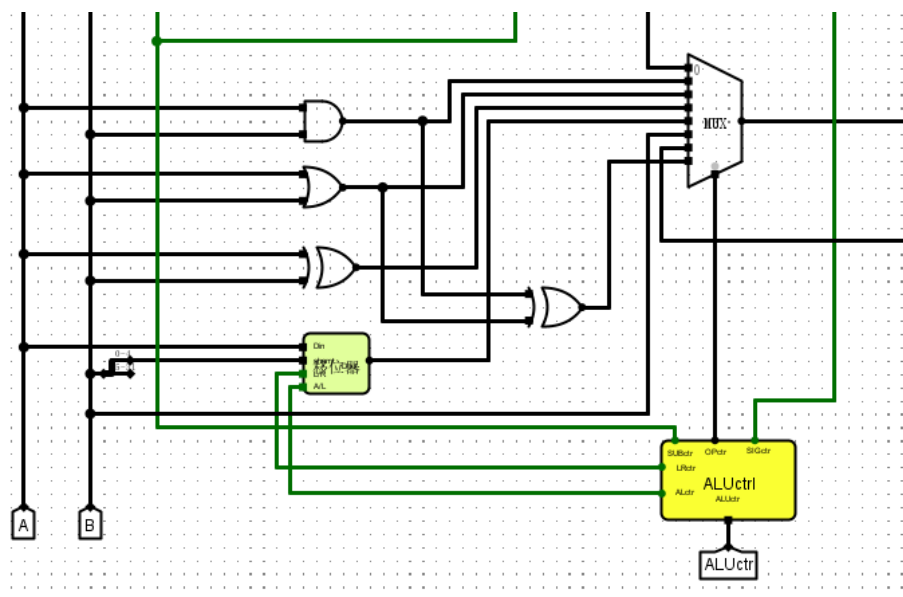
T2的实现电路图在LAB5中

我设计的新指令：一条R型指令，名为aaa，运算效果见下面描述

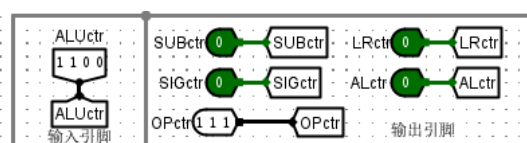


设计的新运算为: $(AB) \text{ xor } (A+B)$

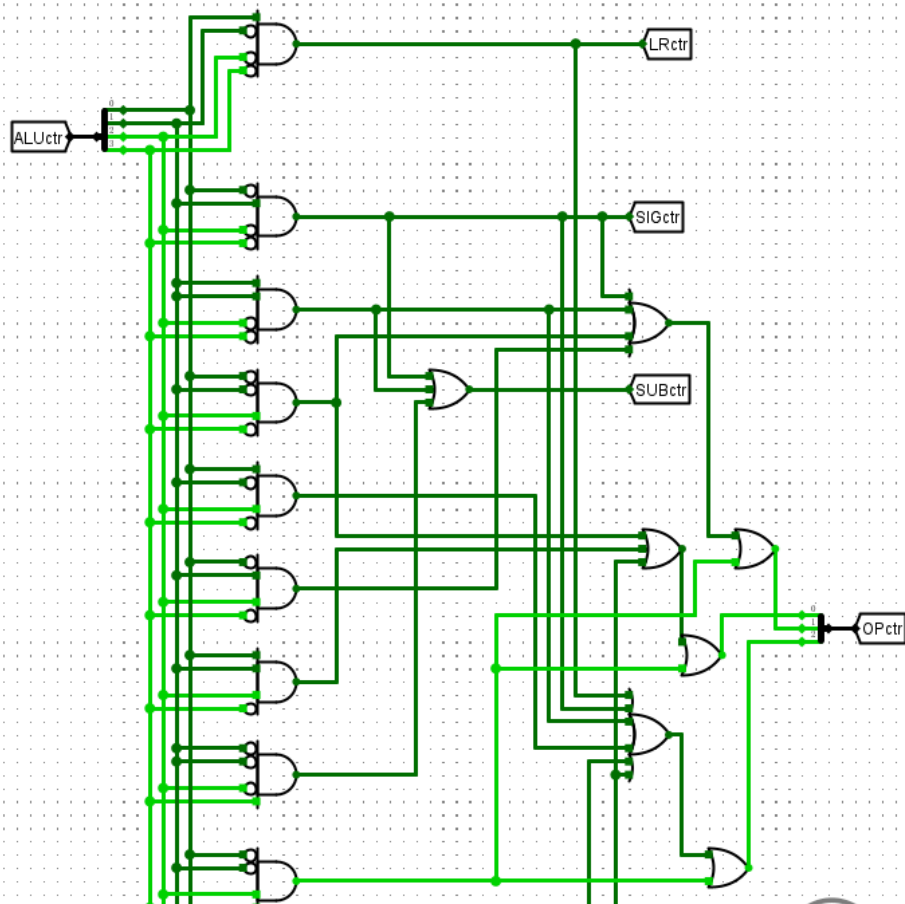
主电路图的修改部分:



在ALUctr中的修改, 以让ALUctr=1100时, OPctr为111, 即选择新运算的结果输出。



ALU操作控制信号生成部件



$A=0111, B=1110, (AB) \text{ xor } (A+B) = 1001$, 正确



1. 初始化，设置被除数为 X ，除数为 Y ，并准备存储商 Q 和余数 R ，其中 Q 初始为 X ， R 初始为 0 。
2. 逐位除法：
 - 从最高位开始（位0到31），每次将余数左移一位，并将被除数的下一位添加到余数中
 - 判断当前余数 R 是否大于或等于除数 Y ：
 - 如果 $R \geq Y$ ，则用 R 减去 Y ，并在商 Q 对应的位置置为1。
 - 如果 $R < Y$ ，则商 Q 对应的位置为0。
3. 左移寄存器 R 和 Q ，重复2中的过程32次，每次处理一个位，直到所有32位都处理完。
4. 输出最终结果，商 Q 存储最终的商，余数 R 存储最终的余数。

每次循环都要对寄存器R和Q实现同步“左移”，左移时，Q的最高位移入R的最低位，Q中空出的最低位被上“商”。从低位开始，逐次把商的各个数位左移到Q中。每次由控制逻辑根据ALU运算结果的符号位来决定上商为0还是1。